Module - 4

05/03/26

Bin Packing Algorithms

Bins - fixed sized containers

objects of varying size.

objective - minimize no: of bins used

Applications - common logistics (loading of trucks)

- resource management
  memory allocation
  file backup system

1. First fit (FF)
2. First Fit decreasing (FFD)
3. Best fit (BF)
4. Next fit (NF)
5. Worst fit (WF)

1. First fit strategy

Step 1) consider items in the given order

Step 2) place each item in the first bin that has enough remaining capacity

Step 3) If not no existing bin can accomodate the item, open a new bin.

Eg: A gym worker lifting up by stacking weights on shelves the weights given in kg are stated below

12, 16, 18, 8, 16, 12, 10, 4, 6, 10, 12, 16, 20, 10, 24

Each shelf has a load capacity of 40kg Find

a) a lower bound for number of shelves needed so that all the weights can be stacked on the shelves

b) use the first fit bin packing algm to find no: of shelves needed to stack all of the weights

Ans: a) Lower Bound (LB) $\geq \frac{\text{Total capacity of all objects}}{\text{Size of each bin}}$

min no: of bins

$$LB \geq \frac{194}{40} = 4.85$$

minimum bin required = 5

b) shelf 1: [12] (28)
shelf 1: [12 | 16] (12)
shelf 2: [18] (22)
shelf 1: [12 | 16 | 8] (4)
shelf 2: [18 | 16] (6)
shelf 3: [12 | 10] (18)
shelf 1: [12 | 16 | 8 | 4] (0)
shelf 2: [18 | 16 | 6] (0)
shelf 3: [12 | 10 | 10] (8)

shelf 4 [12] (28)
shelf 4 [12 | 16] (12)
shelf 5 [20] (20)
shelf 4 [12 | 16 | 10] (2)
shelf 6 [24] (16)

6 shelves are required

First fit decreasing

- It works by sorting items in decreasing order of size. Then placing each item into the first ~~avail~~ available bin that has sufficient capacity.

Q 12 16 18 8 16 12 10 4 6 10 12 16 20 10 24

descending order: 24 20 18 16 16 16 12 12 12 10 10 10 8 6 4

First fit

Shelf 1: [24] (16)
Shelf 2: [20] (20)

shelf 2 | 20 | 18 | (2)

shelf 1 | 24 | 16 | (0)

shelf 3 | 16 | (24)

shelf 3 | 16 | 16 | (8)

shelf 4 | 12 | (28)

shelf 4 | 12 | 12 | (16)

shelf 4 | 12 | 12 | 12 | (4)

shelf 5 | 10 | (30)

shelf 5 | 10 | 10 | (20)

shelf 5 | 10 | 10 | 10 | (10)

shelf 3 | 16 | 16 | 8 | (0)

shelf 5 | 10 | 10 | 10 | 6 | (2)

shelf 4 | 12 | 12 | 12 | 4 | (0)

Full bin Packing Algo

It identifies combinations of objects that will fill a bin & any objects that cannot be grouped to fill a bin, will be packed using the first fit strategy

eg: 12 16 18 8 16 12 10 4 6 10 12 16 20 10 24 → Size = 40

1. 12+18+10
2. 16+8+16
3. 12+12+16
4. 4+6+10+20

⇒ No of bins = 5

rem → 10, 24 → apply first fit

5. 10+24

* Time complexity of first fit = $O(n \log n)$ where n is no of objects

Space complexity of FF = $O(n)$

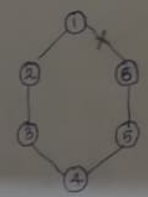
2/02/26

Module - 3

- Greedy strategy
- Spanning tree
- Minimal cost spanning tree
- Prims, Kruskal Algm
- Complexity

Spanning Tree

A spanning tree is defined as a subgraph of a graph having all the vertices and (n-1) edges where n stands for no of vertices

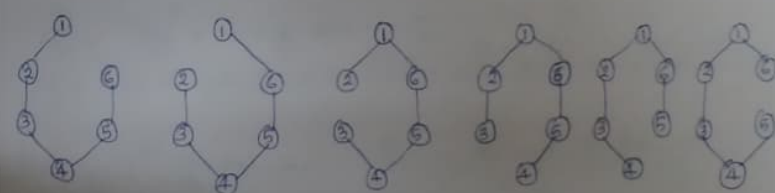


$G = (V, E)$

$V = \{1,2,3,4,5,6\}$

$E = \{(1,2),(2,3),(3,4),(4,5),(5,6),(6,1)\}$

No: of possible spanning tree = ${}^{|E|}C_{|V|-1}$ = ${}^{6}C_{5}$ = ${}^{6}C_{1}$ = 5 → (for 1 cycle)



Spanning trees of given graph

• when graph has more than 1 cycle,

no of spanning trees = ${}^{|E|}C_{|V|-1}$ - No of cycles

eg

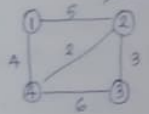
No of spanning trees = ${}^{7}C_5 - 2 = \frac{7!}{5! \times 2!} - 2$

$= \frac{7 \times 6}{2} - 2$

$= 21 - 2 = \underline{\underline{19}}$

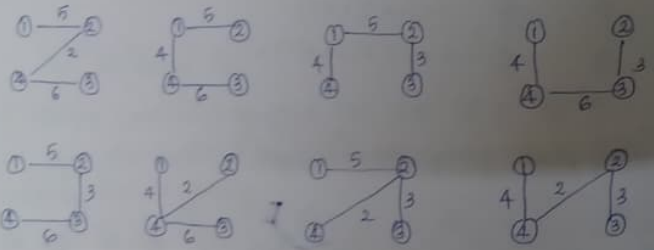
Minimal Cost Spanning Tree

eg



Find which ST has minimum cost

Ans. No of spanning trees = ${}^{5}C_3 - 2 = \frac{5!}{3!\,2!} - 2 = \frac{5 \times 4}{2} - 2 = \underline{\underline{10 - 2 = 8}}$



Min cost sp ⇒

→ Min cost = $\underline{\underline{9}}$

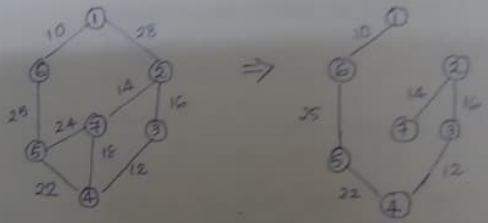
Greedy strategy for Minimal cost

1. Prims Algorithm

i) Identify a min cost edge

ii) Then find the next min cost edge which is connected

eg:



min cost = 10 + 25 + 22 + 12 + 16 + 14

= $\underline{\underline{99}}$

eg:

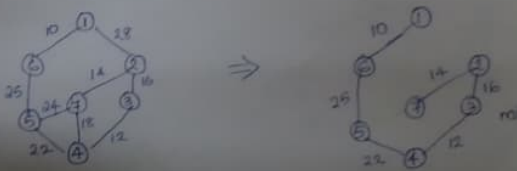
min cost = 3 + 2 + 3 + 4 = $\underline{\underline{12}}$

2. Kruskal's Algorithm

i) Always select a minimum cost edge

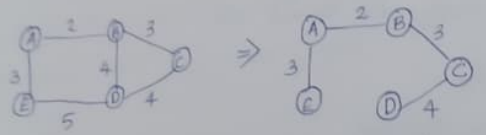
ii) If it forms a cycle discard it that edge and identify the next min cost edge

eg:



min cost = $\underline{\underline{99}}$

eg



min cost = 12

Time complexity

out of $|E|$ edges, we need to select $|V|-1$ edges
↳ neglect

$= \Theta(|V||E|)$

$= \Theta(n \cdot e)$

$\approx \Theta(n^2)$ → worst case

If using a heap (minheap),
deletion of each elem gives min cost node → takes $\log n$
n such deletions are reqd
so, bestcase complexity,
$\Omega(n \log n)$

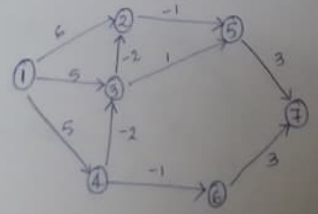
10/8/20

Dynamic Programming

Try out all possible solutions and pick up the best solution.

Bellman-Ford Algorithm

Single-source shortest path



* Select any vertex and find out shortest distance to all other vertex and selected vertex is known as source vertex

Relaxation of edges

Suppose you have an edge b/w (u,v)

if $d[u] + c(u,v) \le d[v]$

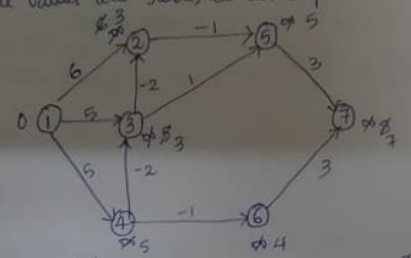
$d[v] = d[u] + c(u,v)$

1) Initialize source vertex as 0, update all other vertices as ∞
2) Prepare the edge list

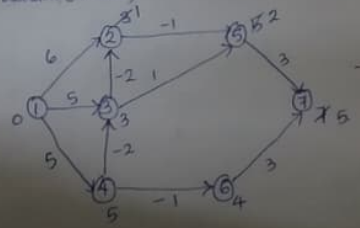
$E = \{(1,2), (1,3), (1,4), (2,5), (3,2), (3,5), (4,3), (4,6), (5,7), (6,7)\}$

3) Relax all edges in the given edge list.
$(n-1)$ times, where n is given by no of vertices.

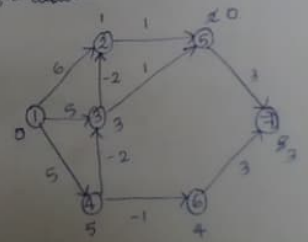
Once values are stable, we can stop.



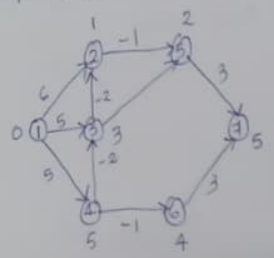
next iteration (2nd one)



3rd iteration



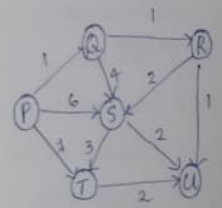
4th iteration



Distance Table

	dist
① → ①	0
① → ②	1
① → ③	3
① → ④	5
① → ⑤	0
① → ⑥	4
① → ⑦	3

Q. Given a weighted directed graph, apply the Bellman ford algm to find the single source shortest path by considering P as a source vertex and show the results after each pass of the algm

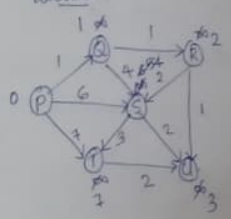


1. Algorithm
2. Initialization step and edge list
3. Iteration process (showing all pass)
4. Final shortest path display

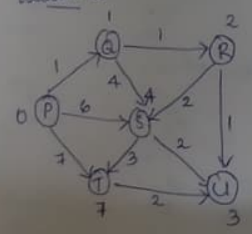
Ans. Edge list

E = {(P,Q), (P,S), (P,T), (Q,R), (Q,S), (R,S), (R,U), (S,T), (S,U), (T,U)}

iteration 1



iteration 2



Distance table

	dist
P → P	0
P → Q	1
P → S	4
P → T	7
P → R	2
P → U	3

Analysis

Time complexity

$O(|E| \cdot |V|-1) = O(|E||V|)$ (approx, remove -1)

let $|E| = |V| = n$,

$O(n \cdot n) = O(n^2)$ → min time reqd (best case)

worst case, complete graph →

if n is no. of vertices,

$|E| = \frac{n(n-1)}{2}$ for complete graph

For directed graph, $|E| = n(n-1)$

complexity

$O(|E||V|-1) = O(n(n-1)n)$

$= O(n^3)$ → worst case complexity

Advantage - works ~~also~~ even with -ve weight edges

Drawbacks

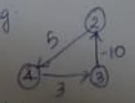
if there is a -ve weighted cycle, the graph can't arrive at the correct answer → graph can't be solved.

↳ value will not stabilise even after max iteration

↓

used to identify if graph contains -ve weight edges

eg.



total weight

= 5+3-10

Dynamic Programming

Optimal Matrix Multiplication

$$\underset{2\times3}{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix} \qquad \underset{3\times2}{B} = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \\ b_{31} & b_{32} \end{bmatrix}$$

$$C = A \times B = \begin{bmatrix} a_{11}\cdot b_{11} + a_{12}\cdot b_{21} + a_{13}\cdot b_{31} & a_{11}\cdot b_{12} + a_{12}\cdot b_{22} + a_{13}\cdot b_{32} \\ a_{21}\cdot b_{11} + a_{22}\cdot b_{21} + a_{23}\cdot b_{31} & a_{21}\cdot b_{12} + a_{22}\cdot b_{22} + a_{22}\cdot b_{32} \end{bmatrix}$$

$\underbrace{\overset{A}{2\times3}\ \overset{B}{3\times2}}_{C = 2\times2}$

$12 = 2\times3\times2$

$\overset{P}{2\times4}\ \overset{Q}{4\times3}$

$P\times Q = 2\times4\times3 = 24$

Suppose we want to multiply 3 matrices $A_1 \times A_2 \times A_3$
it can be done as. $(A_1\times A_2)\times A_3$ or $A_1\times(A_2\times A_3)$

1. $A_{1\,2\times3} \times A_{2\,3\times4} \times A_{3\,4\times2}$

$(A_1\times A_2) \to 2\times4$
$(A_1\times A_2)\times A_3 \to 2\times2$ } on another way ∵ matrix multiplication is associative

tot no. of multiplications $A_1\times A_2$
$2\times3\times4 = 24$

$2\times4\times2 = 16$
$(A_1\times A_2)\times A_3$

$= 24 + 16 = 40$ multiplications

2. In $A_1\times(A_2\times A_3)$

$(A_2\times A_3) \to 3\times2$ no. of multi $= 3\times4\times2 = 24$

$A_1\times(A_2\times A_3) \to 2\times2$ " $= 2\times3\times2 = \underline{12}$

$\underline{36}$

hence optimal in 2nd method

$$\underset{d_0}{2}\ \underbrace{\overset{A_1}{3}\ \times\ \overset{A_2}{3}}_{d_1}\ \underbrace{\overset{A_2}{4}\ \times\ \overset{A_3}{4}}_{d_2}\ \underset{d_3}{2}$$

$(A_1\times A_2)\times A_3$
$2\ 3\ 3\ 4\ \ 4\ 2$

$\underbrace{2\times3\times4}_{24} + 0$

$2\times4\ \ 4\times2$

$2\times4\times2 = 16$

$24 + 16 = 40$

$C[1,2] + C[3,3] + \overset{d_0\ d_2\ d_3}{2\times4\times2}$

$24 + 0 + 16 = 40$

$A_1\times(A_2\times A_3)$
$2\ 3\ 3\ 4\ 4\ 2$

$0 + \underbrace{3\times4\times2}_{24}$

$2\times3\ \ 3\times2$

$2\times3\times2 = 12$

$24 + 12 = 36$

$C[1,1] + C[2,3] + \overset{d_0\ d_1\ d_3}{2\cdot3\cdot2}$

$0 + 24 + 12 = 36$

$$\therefore C[i,j] = \min_{i\le k<j}\{C[i,k] + C[k+1,j] + d_{i-1}\cdot d_k\cdot d_j\}$$

↳ k varies from i to j

If we want to multiply 10 matrices $i=1, j=10$
k varies from 1, 2, ... 9

If we have n matrices, it can be multiplied in $\frac{{}^{2n}C_n}{n+1}$ ways
value of n = no. of matrices − 1

when $n = 3-1$
$= \frac{{}^{2\times2}C_n}{3} = \frac{{}^{4}C_2}{3} = 2$

$n = 4-1$
$= \frac{{}^{2\times3}C_3}{4} = \frac{{}^{6}C_3}{4} = \underline{5}$

matrix chain multiplication

$$A_1 \times A_2 \times A_3 \times A_4$$

$$\underset{d_0}{3} \quad \underbrace{2 \quad 2}_{d_1} \quad \underbrace{4 \quad 4}_{d_2} \quad \underbrace{2 \quad 2}_{d_3} \quad \underset{d_4}{5}$$

we need to find cost of multiplying matrix 1 to 4

$i=1,\ j=4$

$$C[1,4] = \min_{1 \le k < 4} \{ C[i,k] + C[k+1,j] + d_0 \times d_k \times d_4 \}$$

$$C[1,4] = \min \begin{cases} k=1 & C[1,1] + C[2,4] + d_0 \times d_1 \times d_4 \\ k=2 & C[1,2] + C[3,4] + d_0 \times d_2 \times d_4 \\ k=3 & C[1,3] + C[4,4] + d_0 \times d_3 \times d_4 \end{cases}$$

when i, j diff $\Rightarrow$ i, k gets only 1 value, we can directly calculate.

$C[1,1]=0$

$$C[1,2] = \min_{1 \le k < 2} \{ k=1 \{ C[1,1] + C[2,2] + d_0 \times d_1 \times d_2 \} \}$$

$$= 0 + 0 + 3 \times 2 \times 4 = \underline{\underline{24}}$$

	1	2	3	4
1	0	24	28	58
2		0	16	36
3			0	40
4				0

only find the upper triangular matrix -

$\because$ Lower diagonal matrix will be vacant

$$C[2,3] = \min_{2 \le k < 3} k=2 \{ C[2,2] + C[3,3] + d_1 \times d_2 \times d_3 \}$$

$$= 0 + 0 + 2 \times 4 \times 2 = \underline{\underline{16}}$$

$$C[2,4] = \min_{2 \le k < 4} \begin{cases} k=2 & C[2,2] + C[3,4] + d_1 \times d_2 \times d_4 \\ k=3 & C[2,3] + C[4,4] + d_1 \times d_3 \times d_4 \end{cases}$$

$$= \min \begin{cases} k=2 & 0 + 40 + 40 = 80 \\ k=3 & 16 + 0 + 20 = 36 \end{cases} = \underline{\underline{36}}$$

$$C[1,3] = \min_{1 \le k < 3} \begin{cases} k=1 & C[1,1] + C[2,3] + d_0 \times d_1 \times d_3 \\ k=2 & C[1,2] + C[3,3] + d_0 \times d_2 \times d_3 \end{cases}$$

$$= \min \begin{cases} k=1 & 0 + 16 + 12 = 28 \\ k=2 & 24 + 0 + 16 = 40 \end{cases} = 28$$

$$C[1,4] = \min_{1 \le k < 4} \begin{cases} k=1 & C[1,1] + C[2,4] + d_0 \times d_1 \times d_4 \\ k=2 & C[1,2] + C[3,4] + d_0 \times d_2 \times d_4 \\ k=3 & C[1,3] + C[4,4] + d_0 \times d_3 \times d_4 \end{cases}$$

$$= \min \begin{cases} k=1 & 0 + 36 + 30 = 66 \\ k=2 & 24 + 40 + 60 = 124 \\ k=3 & 28 + 0 + 30 = 58 \end{cases}$$

$$= 58$$

k matrix

$i \downarrow$ \ $j \rightarrow$	1	2	3	4
1		1	1	3
2			2	3
3				3
4				

Hence min cost for multiplying A_1, A_2, A_3, A_4 is 58 but which arrangement gives the min cost.

from the k table, we got $k=3 \rightarrow$ here grping is done at 3 in $[1,4]$

$(A_1 \ A_2 \ A_3) A_4$

and in $[1,3]$ $k=1$ hence grping done at 1

$\therefore ((A_1) A_2 A_3) A_4 \rightarrow ((A_1) \times (A_2 \times A_3)) \times A_4$